Experiment 9: Implement a Basic Reinforcement Learning Model (Q-Learning)

Aim

To implement Reinforcement Learning using the Q-Learning algorithm and develop a Neural Network model using TensorFlow to predict students' results (Pass/Fail) based on study hours, attendance, and assignments.

Algorithm

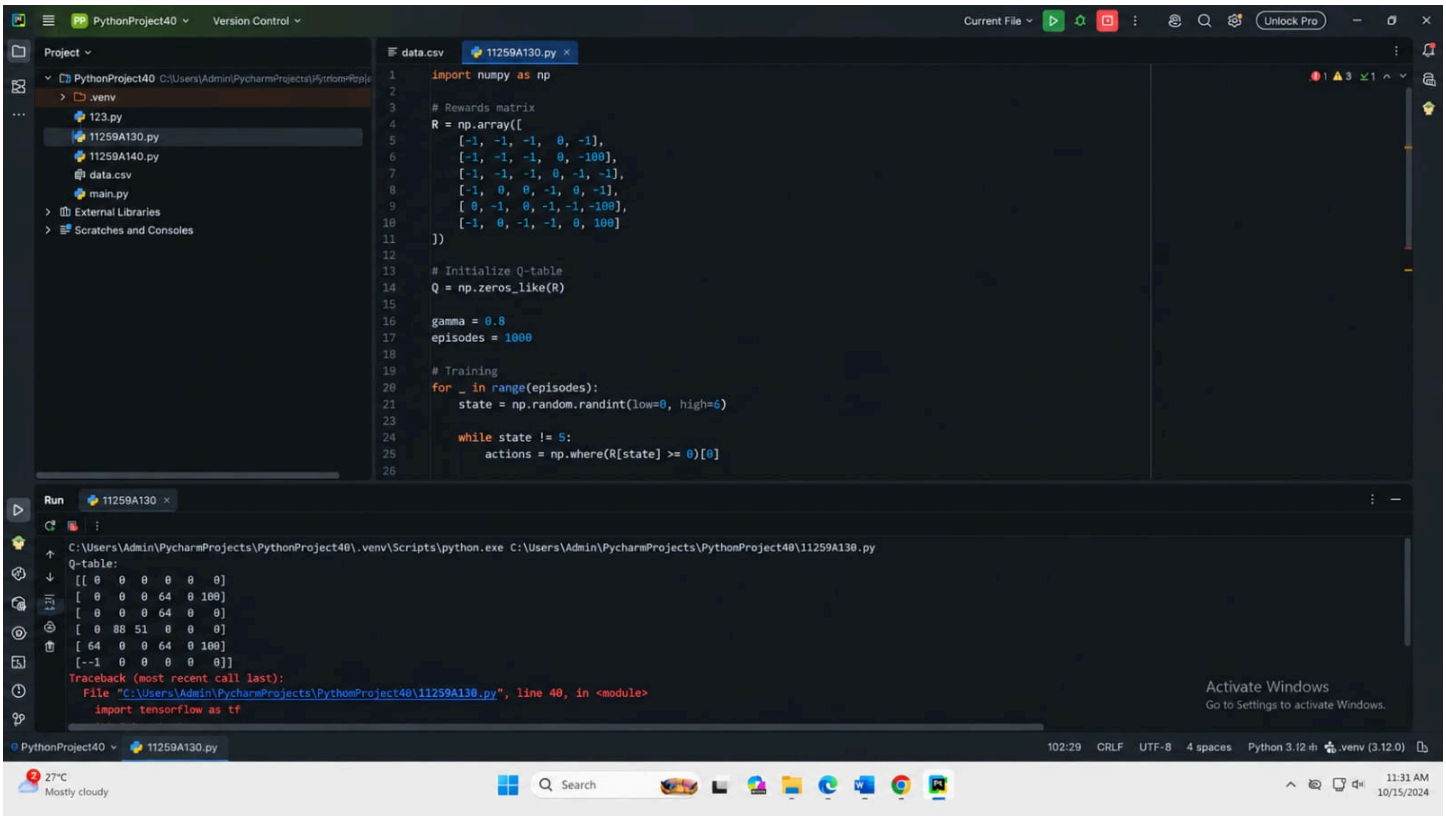
Part A: Q-Learning

1. Import the NumPy library.
2. Define the reward matrix R to represent the rewards for moving between states.
3. Initialize the Q-table with zeros using np.zeros_like(R).
4. Set the discount factor gamma = 0.8 and the number of training episodes to 1000.
5. For each episode, randomly select an initial state.
6. Select a valid action randomly from the available actions in the reward matrix.
7. Update the Q-table using the Q-Learning update rule:

8. \[Q(s,a)=R(s,a)+\gamma\max Q(s',a) \]
9. Continue until the target state (state 5) is reached.
10. Display the final Q-table.

Part B: Neural Network Classification

1. Import Pandas, TensorFlow, and the required Scikit-learn functions.
2. Load the dataset from data.csv.
3. Select Hours_Studied, Attendance, and Assignments as input features.
4. Select Result as the target variable and convert Fail to 0 and Pass to 1.
5. Split the dataset into 80% training data and 20% testing data.
6. Apply StandardScaler to normalize the input features.
7. Build a neural network with two hidden layers containing 8 and 4 neurons, using ReLU activation.
8. Add an output layer with one neuron and sigmoid activation for binary classification.
9. Compile the model using the Adam optimizer, binary cross-entropy loss, and accuracy metric.
10. Train the model for 50 epochs, using 20% of the training data for validation.
11. Predict the results for the testing dataset.
12. Convert predicted probabilities into class labels using a threshold of 0.5.
13. Calculate and display the model's accuracy using accuracy_score().



Result

Part A — Q-Learning: The program trains the Q-table using the reward matrix, a discount factor of 0.8, and 1000 episodes. The resulting Q-table represents the learned action values for moving between states toward the target state (state 5).

Part B — Neural Network: The TensorFlow neural network is trained to classify students' results as Pass or Fail. The program displays the training progress and the final test accuracy.

Sample output for the neural network: